

Memoria Técnica: Control Avanzado de

Turtlesim

Práctica 1 — Robótica (ROS 2)

Centro: E.T.S.I. de Sistemas Informáticos (UPM)

Titulación: Grado en Ingeniería de Computadores

Asignatura: Robótica

Entorno: ROS 2 Jazzy Jalisco / Linux

Paquete Desarrollado: `turtle_control`

1. Introducción y Objetivos

El objetivo de la presente práctica es diseñar e implementar una aplicación en **ROS 2** que extienda las funcionalidades básicas del simulador `turtlesim`. Frente a los nodos convencionales de teleoperación (como `turtle_teleop_key`), este desarrollo incorpora capacidades avanzadas de maniobra, interacción con servicios del simulador, gestión dinámica de parámetros y automatización del despliegue.

Los requisitos establecidos en el enunciado son:

- **Movimiento continuo y simultáneo:** Control mediante las teclas `W` (avance), `S` (retroceso), `A` (giro izquierda) y `D` (giro derecha), garantizando que la tortuga sea capaz de avanzar y rotar al mismo tiempo (curvas continuas y fluidas).
- **Control de trazado (Pen):** Activar y desactivar la pluma de dibujo mediante la tecla `ESPACIO` mediante el servicio `/turtle1/set_pen`.
- **Limpieza de pantalla:** Borrado completo de los trazos existentes al pulsar la tecla `C` utilizando el servicio `/clear`.
- **Reinicio orientado:** Reposicionar la tortuga en el centro exacto de la ventana mirando

hacia arriba (orientación cenital / 90°) mediante la tecla R y el servicio `/turtle1/teleport_absolute`.

- **Parámetros configurables en caliente:** Definición de parámetros para el nivel de log y para la velocidad lineal/angular, con capacidad de alteración en tiempo de ejecución sin reinicio del nodo.
- **Lanzador automático (Launch File):** Archivo `.launch.py` que inicialice simultáneamente el simulador y el nodo de control gestionando argumentos.

2. Arquitectura de la Solución

La aplicación se compone de dos nodos principales que colaboran dentro del grafo de computación de ROS 2:

- `turtlesim_node`: Nodo estándar proporcionado por el paquete `turtlesim`, responsable del motor físico básico y la renderización en interfaz gráfica.
- `teleop_turtle`: Nodo personalizado desarrollado en Python bajo el paquete `turtle_control`, encargado de la captura asíncrona de eventos de teclado, cómputo cinemático diferencial y llamadas a servicios.

Canal / Interfaz	Nombre	Tipo de Mensaje / Servicio	Propósito
Topic (Publisher)	<code>/turtle1/cmd_vel</code>	<code>geometry_msgs/msg/ Twist</code>	Envío continuo de velocidades lineal y angular (30 Hz).
Service Client	<code>/turtle1/set_pen</code>	<code>turtlesim/srv/SetPen</code>	Habilita o deshabilita la escritura de la pluma (<code>off: 0/1</code>).
Service Client	<code>/clear</code>	<code>std_srvs/srv/Empty</code>	Elimina todas las líneas dibujadas sobre el lienzo.
Service Client	<code>/turtle1/ teleport_absolute</code>	<code>turtlesim/srv/ TeleportAbsolute</code>	Mueve la tortuga a <code>(5.54, 5.54)</code> con <code>theta = $\pi/2$</code> (apuntando arriba).

3. Implementación Detallada

3.1. Captura de Teclado y Movimiento Simultáneo

La limitación habitual del teleoperador estándar radica en que el buffer de terminal bloqueante lee caracteres aislados, perdiendo la noción de pulsación mantenida. Para superarlo, se ha integrado la librería `pynput.keyboard` en un hilo secundario independiente (daemon thread).

El nodo mantiene en memoria un conjunto dinámico (`set`) con las teclas actualmente presionadas:

- Al registrarse el evento `on_press`, si la tecla corresponde a `w`, `a`, `s` o `d`, se incorpora al conjunto.
- Al recibirse el evento `on_release`, dicha tecla es retirada de la colección.

Un temporizador periódico de ROS 2 (`create_timer(1/30)`) calcula en cada ciclo las componentes del mensaje `Twist`:

```
def timer_callback(self):  
  
    twist = Twist()  
  
    linear = 0.0  
  
    angular = 0.0  
  
  
  
    if 'w' in self.pressed_keys: linear += self.speed  
  
    if 's' in self.pressed_keys: linear -= self.speed  
  
    if 'a' in self.pressed_keys: angular += self.angular_speed  
  
    if 'd' in self.pressed_keys: angular -= self.angular_speed  
  
  
    twist.linear.x = float(linear)  
  
    twist.angular.z = float(angular)
```

```
self.cmd_vel_pub.publish(twist)
```

De este modo, cuando el usuario presiona simultáneamente `w` y `A`, tanto `linear.x` como `angular.z` son distintos de cero, produciendo un arco circular continuo en lugar de movimientos discretos alternados.

3.2. Clientes de Servicio Asíncronos

Para no bloquear el bucle de eventos principal (ni la publicación del temporizador), todas las invocaciones a los servicios de Turtlesim se realizan de forma asíncrona mediante el método `call_async()`:

- **Control de Pluma (ESPACIO):** Mantiene un flag booleano `pen_enabled`. Al pulsar espacio, se conmuta su valor y se envía una solicitud a `/turtle1/set_pen` con `req.off = 0` (si está habilitado) o `req.off = 1` (si está deshabilitado).
- **Limpieza de Trazos (C):** Envía un mensaje `std_srvs/srv/Empty.Request()` al servicio `/clear`, dejando el fondo azul totalmente libre de líneas.
- **Reinicio Central Cental (R):** El comando `/reset` nativo de Turtlesim restaura la orientación hacia la derecha (0 rad). Para dar cumplimiento estricto al requisito de orientar la tortuga "*mirando hacia arriba*", se invoca `/turtle1/teleport_absolute` fijando las coordenadas al centro del lienzo (`x=5.5444`, `y=5.5444`) y el ángulo a `theta =  $\pi / 2$` radianes (1.57079 rad, equivalente a 90°).

3.3. Parámetros Dinámicos en Tiempo de Ejecución

Se han declarado formalmente en el constructor tres parámetros ROS: `speed` (velocidad lineal), `angular_speed` (velocidad de giro) y `log_level`.

Mediante el mecanismo oficial `self.add_on_set_parameters_callback(...)`, el nodo escucha activamente las modificaciones enviadas externamente (por ejemplo con `ros2 param set`). Al recibirse un cambio de `speed` o `log_level`, este se aplica en caliente de manera inmediata sin necesidad de detener la ejecución.

3.4. Automatización con Launch File

El archivo `launch/turtlesim_teleop.launch.py` orquesta la ejecución completa de la práctica. Declara argumentos configurables (`DeclareLaunchArgument`) para velocidad y logging, e instancia simultáneamente tanto el nodo `turtlesim_node` como el nodo de control `teleop_turtle`,

enlazando sus salidas estándar en consola (`emulate_tty=True`).

4. Guía de Compilación y Ejecución

4.1. Compilación del Espacio de Trabajo

Desde la raíz del espacio de trabajo ROS 2:

```
colcon build --packages-select turtle_control
source install/setup.bash
```

4.2. Lanzamiento de la Aplicación

```
# Lanzamiento estándar con valores por defecto:
ros2 launch turtle_control turtlesim_teleop.launch.py

# Lanzamiento personalizado con velocidad incrementada y log detallado:
ros2 launch turtle_control turtlesim_teleop.launch.py speed:=4.0 log_level:=DEBUG
```

4.3. Modificación Dinámica en Caliente

Desde una terminal secundaria con el entorno activado:

```
# Modificar la velocidad lineal a 6.0:
ros2 param set /teleop_turtle speed 6.0

# Habilitar logging detallado de velocidad:
ros2 param set /teleop_turtle log_level DEBUG
```

5. Validación de Requisitos

Requisito del Enunciado	Estado	Mecanismo Técnico Implementado
Movimiento con teclas W, S, A, D	Cumplido	Mapeo de teclas lineales y angulares sobre <code>geometry_msgs/Twist</code> .
Girar y avanzar simultáneamente	Cumplido	Seguimiento de conjunto de teclas con <code>pynput</code> y temporizador 30 Hz.
Tecla SPACE para habilitar/deshabilitar pintado	Cumplido	Cliente asíncrono para <code>/turtle1/set_pen</code> con flag conmutable.
Tecla C para borrar trazos	Cumplido	Cliente asíncrono para <code>/clear</code> (tipo <code>Empty</code>).
Tecla R para reiniciar al centro y mirando arriba	Cumplido	Cliente asíncrono para <code>/turtle1/teleport_absolute</code> ($x=5.54$, $y=5.54$, $\theta=\pi/2$).
Launch file integrado	Cumplido	Script <code>turtlesim_teleop.launch.py</code> con paso de argumentos.
Parámetro de nivel de logging	Cumplido	Parámetro <code>log_level</code> con callback dinámico de severidad.
Parámetro de velocidad en tiempo de ejecución	Cumplido	Parámetro <code>speed</code> con actualización instantánea mediante <code>ros2 param set</code> .

6. Conclusiones

El desarrollo realizado cumple de manera rigurosa con todas las especificaciones solicitadas en el enunciado. La utilización de listeners asíncronos y temporizadores periódicos ha permitido superar las limitaciones de los controladores tradicionales de consola, logrando un control suave, intuitivo y concurrente sobre el simulador. Asimismo, la arquitectura adoptada pone en práctica los conceptos nucleares de ROS 2: tópicos continuos, llamadas a servicios, parámetros reactivos y archivos de lanzamiento estructurados.